BPL_TEST2_Perfusion - demo

This notebook explore perfusion cultivation in comparison with ordinary continuous cultivation (chemostat) and use comparable settings to earlier notebook. Further you see here examples of interaction with the simplified commands par(), init(), simu() etc as well as direct interaction with the FMU which is called "model" here. The last simulation is always available in the workspace and called "sim_res". Note that describe() brings mainly up from descriptive information from the Modelica code from the FMU but is complemented by some information given in the Python setup file.

In [1]:
```
# Import the application model and context variables
from BPL_TEST2_Perfusion_fmpy import*
```

```
Windows - run FMU pre-compiled JModelica 2.14
```

In [2]:
```
# Import the general FMU_explore functions ana adapt them to the application
from fmu_explore_fmpy import AdaptWith

Application = AdaptWith(parValue, parLocation, parCheck, keyVariables, \
                        model_description, fmu_model, fmu_process_diagram, \
                        MSL_usage, MSL_version, BPL_version, \
                        opts_std, simulationTime, timeDiscreteStates, \
                        diagrams, ax, lines, external_function = cstrProdMax)

par = Application.par
init = Application.init
disp = Application.disp
simu = Application.simu
show = Application.show

model_get = Application.model_get
resetPen = Application.resetPen
process_diagram = Application.process_diagram
describe_general = Application.describe_general
describe_parts = Application.describe_parts
describe_MSL = Application.describe_MSL

FMU_explore_info = Application.FMU_explore_info
system_info = Application.system_info
SDG = Application.SDG
```

In [3]:
```
run -i BPL_TEST2_Perfusion_fmpy_explore.py
```

```
Model for the process has been setup. Key commands:
 - par()       - change of parameters and initial values
 - init()      - change initial values only
 - simu()      - simulate and plot
 - newplot()   - make a new plot
 - show()      - show plot from previous simulation
 - disp()      - display parameters and initial values from the last simulation
 - describe()  - describe culture, broth, parameters, variables with values/units

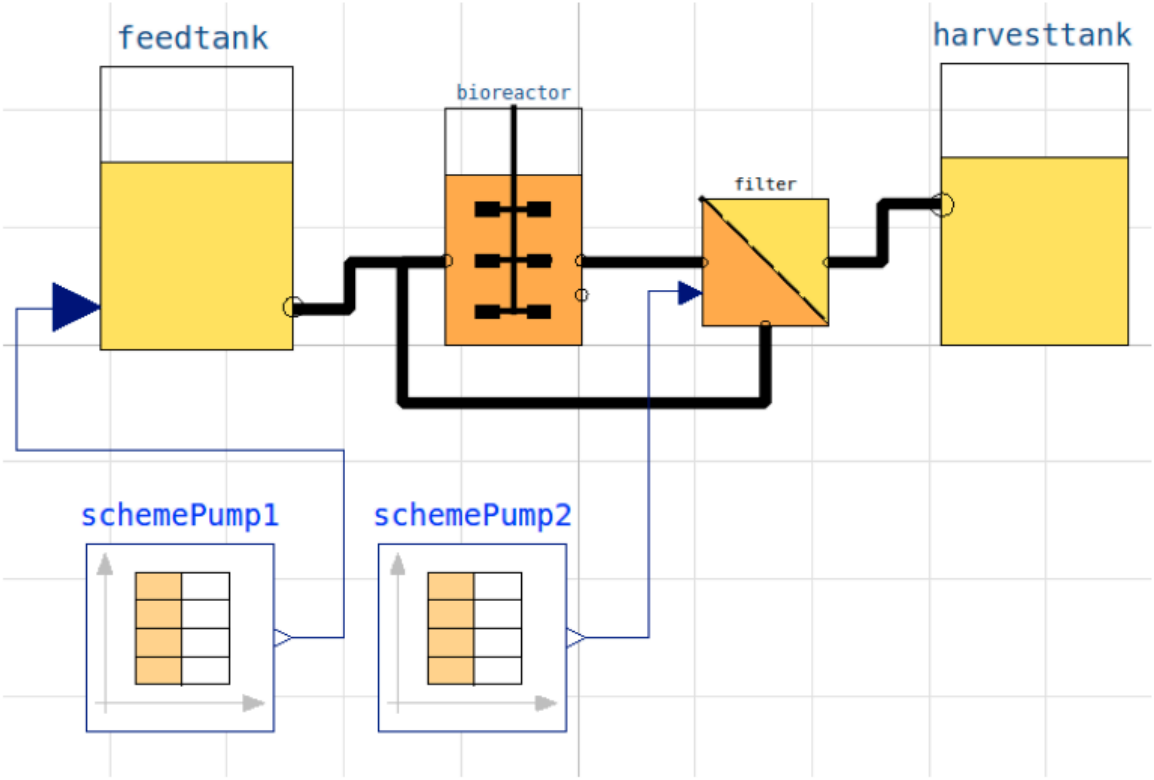
Note that both disp() and describe() takes values from the last simulation
and the command process_diagram() brings up the main configuration

Brief information about a command by help(), eg help(simu)
Key system information is listed with the command system_info()
```

In [4]:
```
%matplotlib inline
plt.rcParams['figure.figsize'] = [25/2.54, 20/2.54]
```

In [5]:
```
process_diagram()
```

```
No processDiagram.png file in the FMU, but try the file on disk.
```

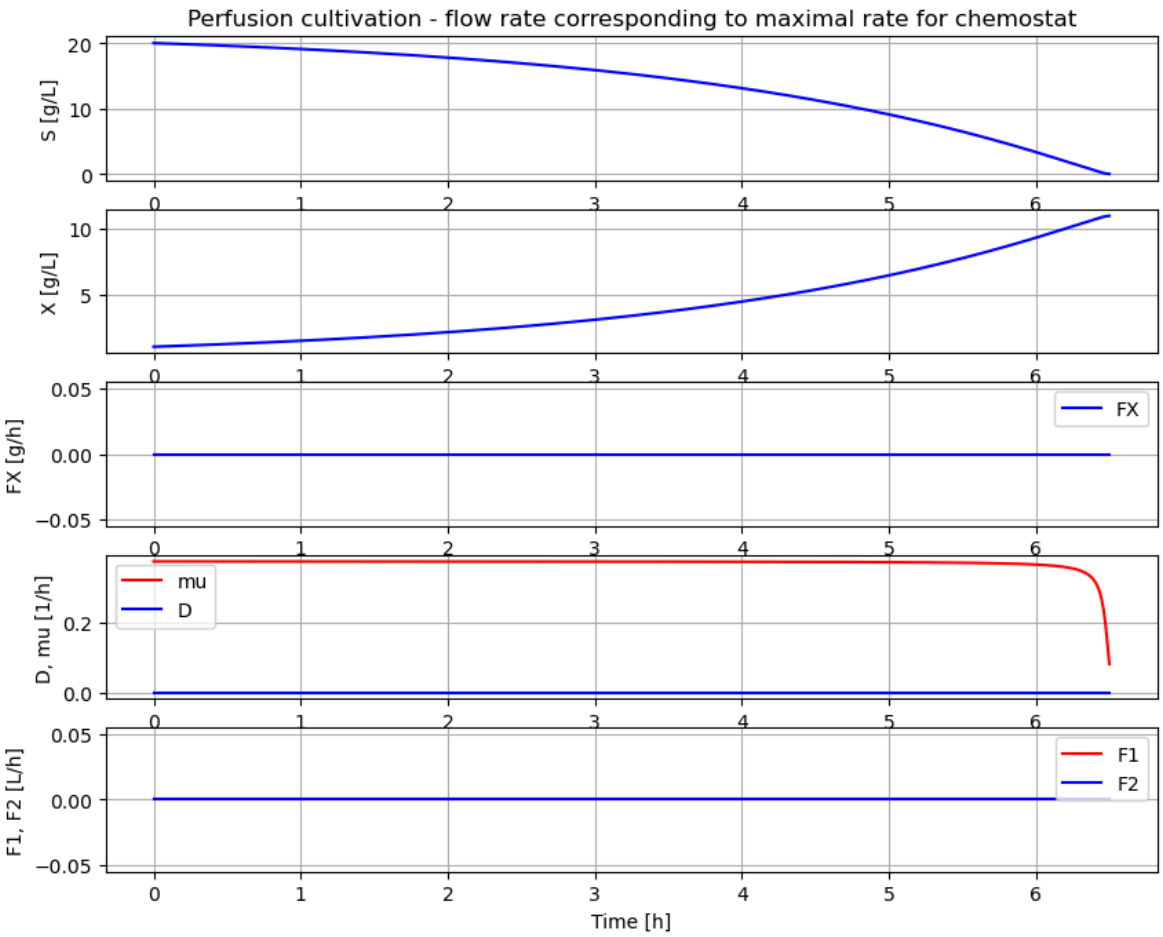


In [6]:
```
# Process parameters used throughout
par(Y=0.5, qSmax=0.75, Ks=0.1)                                    # Culture
par(filter_eps=0.10, filter_alpha_X=0.02, filter_alpha_S=0.10)    # Filter
par(S_in=30.0)                                                    # Inlet subs
init(V_start=1.0, VX_start=1.0)                                   # Process in
eps = parValue['filter_eps']                                       # Pump sche
```

In [7]:
```
# Simulation of process with flow rate clot to wash-out for chemostat

init(VS_start=20)                                    # Process initial
par(pump1_t1=10, pump2_t1=10)                        # Pump schedule - recyc
par(pump1_F1=2.5*0.155, pump2_F1=2.5*0.155/eps)
par(pump1_t2=940, pump2_t2=940, pump1_t3=950, pump2_t3=950, pump1_t4=960, pump2_
```

```
newplot(title='Perfusion cultivation - flow rate corresponding to maximal rate f
simu(6.5)
```



In [8]:
```
# Concentration factor of the filter
c=model_get('filter.retentate.c[1]')/model_get('filter.inlet.c[1]')
print('Conc factor of perfusion filter =', np.round(c,3))
```

```
Conc factor of perfusion filter = 1.186
```

In [9]:
```
sim_res = Application.sim_res
c_data=sim_res['filter.retentate.c[1]']/sim_res['filter.inlet.c[1]']
print('Conc factor variation', np.round(min(c_data[151:]), 3),'to', np.round(max
```

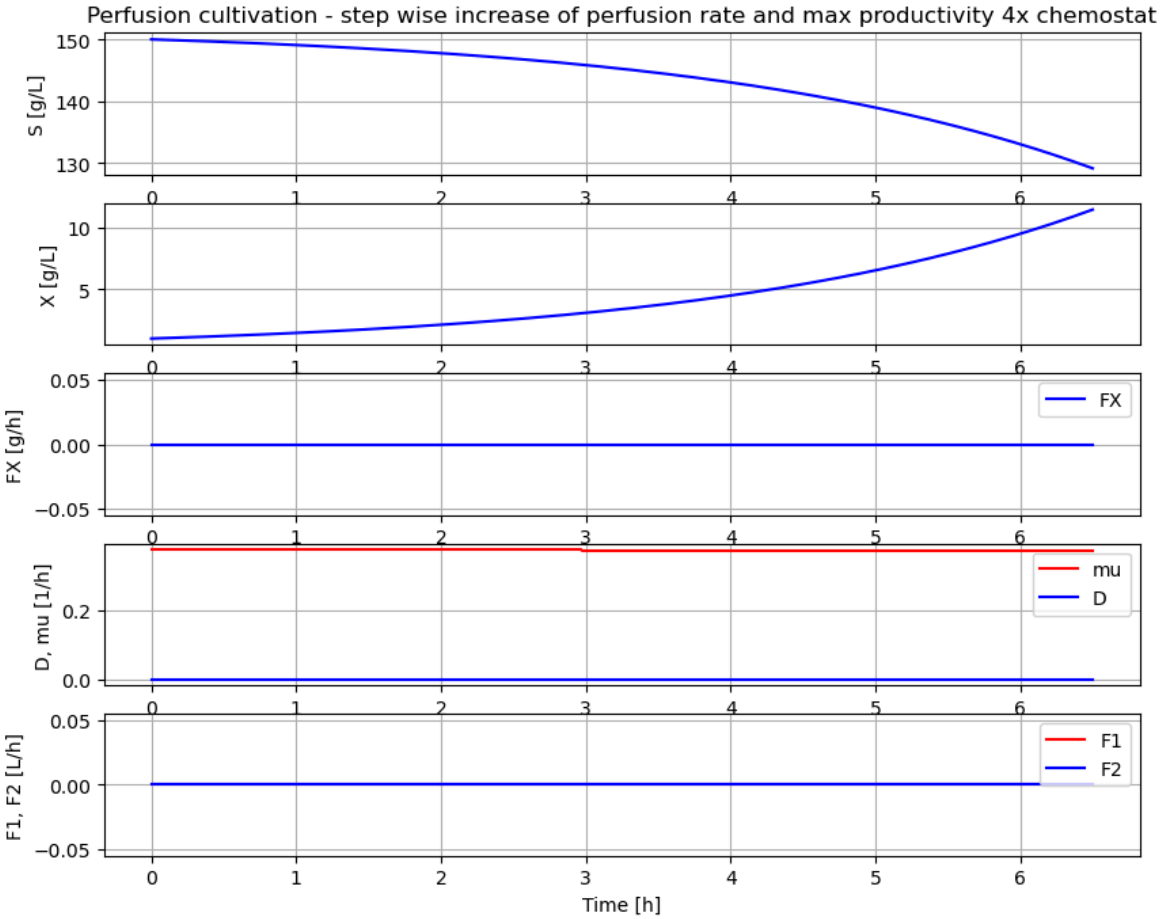
```
Conc factor variation 1.186 to 1.186
```

In [10]:
```
# Simulation of process with step-wise increase of pefusion rate until wash-out.
# This means that re-circulation rate change at the same time as the perfusion r

init(VS_start=150)                                  # Process initial varie

par(pump1_t1=12, pump2_t1=12)                       # Pump schedule - recyc
par(pump1_F1=2.5*0.155, pump2_F1=2.5*0.155/eps)
par(pump1_t2=22, pump2_t2=22)
par(pump1_F2=2.5*0.35, pump2_F2=2.5*0.35/eps)
par(pump1_t3=32, pump2_t3=32)
par(pump1_F3=2.5*0.63, pump2_F3=2.5*0.63/eps)
par(pump1_t4=42, pump2_t4=42)
par(pump1_F4=2.5*0.83, pump2_F4=2.5*0.83/eps)

newplot(title='Perfusion cultivation - step wise increase of perfusion rate and
simu(6.5)
```



In [11]:
```
# Simulation without a plot and just to check typical values at high production
simu(6.5)
#c_data=sim_res['filter.retentate.c[1]'][304:]/sim_res['filter.inlet.c[1]'][304:
#print('Conc factor variation', np.round(min(c_data[304:]), 3), 'to', np.round(m
```

In [12]:
```
#describe('cstrProdMax')
```

In [13]:
```
# The maximal biomass productivity before washout is obtained aroudn 40 hours
model_get = Application.model_get
np.round(model_get('harvesttank.inlet.F')*model_get('harvesttank.inlet.c[1]'),1)
```

Out[13]: np.float64(0.0)

In [14]:
```
# Thus perfusion (with this filter) brings a productivity improvement of about
np.round(23.5/5.6,1)
```

Out[14]: np.float64(4.2)

In [15]:
```
# Finally we check the filter flow rates at time 40 hour - note the negative sig
model_get('filter.inlet.F')
```

Out[15]: 0.0

In [16]:
```
model_get('filter.filtrate.F')
```

Out[16]: -0.0

In [17]:
```
model_get('filter.retentate.F')
```

Out[17]: -0.0

Summary

- The perfusion filter had a concentration factor of cells around 1.08 and re-cycling flow was set to a factor 10 higher than the perfusion rate and changed when perfusion rate was change to keep the ratio factor 10.
- The first simulation showed that by cell retention using perfusion filter the process could be run at a perfusion flow rate at the maximal flow rate possible for corresponding chemostat culture and cell concetration increased steadily.
- The second simulation showed that with a proper startup cell concentration, the cell concentration remained constant when perfusion rate increased in a similar way as what we see in a chemostat.
- The second simulation also showed that biomass productivity in this case was increased by a factor 4.2 compared to chemostat.
- If the perfusion rate increased to higher levels washout started but the decrase of cell concentration was slow.

Some of you who read this may have your perfusion experience with CHO-cultures. For such cultures the cell concentration do increase with increase of perfusion rate and there are understood reasons for that. But for this simplifed process as well as microbial processes they typically keep cell concentration constant when flow rate is chaged, and that under quite wide conditions. I will try come back to this phenomena in a later notebook.

Appendix

In [18]:
```
disp('culture')
```

```
Y : 0.5
qSmax : 1.0
Ks : 0.1
```

In [19]:
```
describe('mu')
```

```
Cell specific growth rate variable : 0.375 [ 1/h ]
```

In [20]:
```
# List of components in the process setup and also a couple of other things like
describe('parts')
```

```
['bioreactor', 'bioreactor.culture', 'D', 'feedtank', 'filter', 'harvesttank', 'M
SL', 'schemePump1', 'schemePump2']
```

In [21]:
```
describe('MSL')
```

```
MSL: RealInput, RealOutput, CombiTimeTable, Types
```

In [22]:
```
system_info()
```

```
System information
 -OS: Windows
 -Python: 3.12.11
 -Scipy: 1.16.0
 -FMPy: 0.3.27
 -FMU by: JModelica.org
 -FMI: 2.0
 -Type: CS
 -Name: BPL.Examples_TEST2.Perfusion
 -Generated: 2025-07-26T09:41:00
 -MSL: 3.2.2 build 3
 -Description: Bioprocess Library version 2.3.1
 -Interaction: FMU_explore version 1.2.4
```

In [ ]: